

Backend API and C++ Binding Contract

This document is the handoff contract between:

- React frontend
- Python FastAPI layer
- C++ core engine exposed to Python through `pybind11`

The frontend should call FastAPI only. FastAPI should call a small set of Python wrapper functions. Those Python wrappers should call the C++ functions exposed by `pybind`.

Three Clusters

Split the work into these three clusters:

1. Auth and Users
2. Products, Cart, and Dashboard
3. Interactions and Recommendations

Each cluster below includes:

- HTTP API endpoint
- request body
- response body
- Python wrapper function
- C++ function that the C++ team should expose for `pybind`

Quick API To C++ Signature Matrix

This is the main list your C++ team needs. Every FastAPI endpoint should call one Python wrapper, and every Python wrapper should call one of these C++ functions through `pybind`.

Cluster 1: Auth and Users

FastAPI endpoint	C++ function needed
<code>POST /auth/login</code>	<code>AuthResult authenticate_user(const std::string& email, const std::string& password);</code>
<code>POST /auth/signup</code>	<code>AuthResult create_user(const UserInput& input);</code>
<code>GET /users/{user_id}</code>	<code>UserDTO get_user_by_id(int user_id);</code>

Cluster 2: Products, Cart, and Dashboard

FastAPI endpoint	C++ function needed
<code>GET /products</code>	<code>std::vector<ProductDTO> list_products(const std::string& category);</code>
<code>GET /products/{item_id}</code>	<code>ProductDTO get_product_by_id(int item_id);</code>
<code>POST /products</code>	<code>ProductDTO create_product(const ProductInput& input);</code>
<code>PUT /products/{item_id}</code>	<code>ProductDTO update_product(int item_id, const ProductInput& input);</code>
<code>DELETE /products/{item_id}</code>	<code>bool delete_product(int item_id);</code>
<code>GET /users/{user_id}/cart</code>	<code>std::vector<CartItemDTO> get_cart(int user_id);</code>
<code>POST /users/{user_id}/cart</code>	<code>bool add_to_cart(int user_id, int item_id, int quantity);</code>
<code>DELETE /users/{user_id}/cart/{item_id}</code>	<code>bool remove_from_cart(int user_id, int item_id);</code>
<code>POST /users/{user_id}/checkout</code>	<code>CheckoutResult checkout(int user_id);</code>
<code>GET /dashboard/stats</code>	<code>DashboardStats get_dashboard_stats();</code>

Cluster 3: Interactions and Recommendations

FastAPI endpoint	C++ function needed
POST /interactions	InteractionDTO record_interaction(int user_id, int item_id, const std::string& interaction_type);
GET /interactions/recent	std::vector<InteractionDTO> get_recent_interactions(int limit);
GET /recommendations/{user_id}	std::vector<RecommendationDTO> get_recommendations(int user_id, int limit);
GET /products/{item_id}/related	std::vector<RelatedProductDTO> get_related_products(int item_id, int limit);
GET /trending	std::vector<ProductDTO> get_trending(int limit);

DTO Types Needed By C++

These return/input types can be plain structs. Pybind can expose them as classes, or the binding layer can convert them into Python dictionaries.

```

struct UserInput {
    std::string name;
    std::string email;
    std::string password;
    std::string category;
    std::string region;
};

struct UserDTO {
    int id;
    std::string name;
    std::string email;
    std::string category;
    double score;
    std::string region;
    int activity_score;
    std::string level;
};

struct AuthResult {
    bool success;
    std::string error;
    UserDTO user;
    std::string token;
};

struct ProductInput {
    std::string name;
    double price;
    std::string category;
    int stock;
    std::string image;
    std::string description;
    double popularity_score;
};

struct ProductDTO {
    int id;
    std::string name;
    double price;
    std::string category;
    double popularity_score;
    int stock;
    std::string image;
    std::string description;
};

```

```
struct InteractionDTO {
    int id;
    int user_id;
    int item_id;
    std::string type;
    std::string timestamp;
};

struct CartItemDTO {
    int user_id;
    int item_id;
    int quantity;
    ProductDTO product;
};

struct CheckoutResult {
    bool success;
    int purchased;
};

struct DashboardStats {
    int totalProducts;
    int totalUsers;
    int totalInteractions;
    std::string mostPopularCategory;
};

struct RecommendationDTO {
    ProductDTO product;
    double recommendation_score;
    std::string reason;
};

struct RelatedProductDTO {
    ProductDTO product;
    double similarity_score;
};
```

Shared Data Shapes

User

```
{
  "id": 1,
  "name": "Mariam Saleh",
  "email": "mariam@biteapple.test",
  "category": "Electronics",
  "score": 92,
  "region": "Cairo",
  "activity_score": 8,
  "level": "Normal"
}
```

Do not return `password` or `password_hash` to the frontend.

Product

```
{
  "id": 101,
  "name": "Wireless Keyboard",
  "price": 1650,
  "category": "Electronics",
  "popularity_score": 91,
  "stock": 12,
  "image": "https://example.com/image.jpg",
  "description": "Low-profile wireless keyboard."
}
```

Interaction

```
{
  "id": 1,
  "user_id": 1,
  "item_id": 101,
  "type": "view",
  "timestamp": "2026-05-05T10:30:00Z"
}
```

Allowed interaction types:

- view
- click
- cart
- purchase

Python should translate these to C++ enum values:

- view -> VIEW
- click -> CLICK
- cart -> ADD_TO_CART
- purchase -> PURCHASE

Cart Item

```
{
  "user_id": 1,
  "item_id": 101,
  "quantity": 2,
  "product": {
    "id": 101,
    "name": "Wireless Keyboard",
    "price": 1650,
    "category": "Electronics",
    "popularity_score": 91,
    "stock": 12,
    "image": "https://example.com/image.jpg",
    "description": "Low-profile wireless keyboard."
  }
}
```

Cluster 1: Auth and Users

Owner focus: HashMap-backed user store and authentication helpers.

POST /auth/login

Request:

```
{
  "email": "mariam@biteapple.test",
  "password": "password123"
}
```

Response:

```
{
  "user": {
    "id": 1,
    "name": "Mariam Saleh",
    "email": "mariam@biteapple.test",
    "category": "Electronics",
    "score": 92,
    "region": "Cairo"
  },
  "token": "dev-token-1"
}
```

Python wrapper:

```
def login_user(email: str, password: str) -> dict:
    return cpp_core.authenticate_user(email, password)
```

C++ function to expose:

```
AuthResult authenticate_user(const std::string& email,
                             const std::string& password);
```

Expected behavior:

- Return success with public user data when credentials are valid.
- Return failure/error when credentials are invalid.
- Never expose password fields.

POST /auth/signup

Request:

```
{
  "name": "Nour Adel",
  "email": "nour@example.com",
  "password": "password123",
  "category": "Electronics",
  "region": "Cairo"
}
```

Response:

```
{
  "user": {
    "id": 4,
    "name": "Nour Adel",
    "email": "nour@example.com",
    "category": "Electronics",
    "score": 50,
    "region": "Cairo"
  },
  "token": "dev-token-4"
}
```

Python wrapper:

```
def signup_user(payload: dict) -> dict:
    return cpp_core.create_user(payload)
```

C++ function to expose:

```
AuthResult create_user(const UserInput& input);
```

Expected behavior:

- Reject duplicate email.
- Create a new user ID.
- Set default score to 50 if no score is supplied.
- Set activity score to 0 .
- Set level to Normal .

GET /users/{user_id}

Response:

```
{
  "id": 1,
  "name": "Mariam Saleh",
  "email": "mariam@biteapple.test",
  "category": "Electronics",
  "score": 92,
  "region": "Cairo",
  "activity_score": 8,
  "level": "Normal"
}
```

Python wrapper:

```
def get_user(user_id: int) -> dict:
    return cpp_core.get_user_by_id(user_id)
```

C++ function to expose:

```
UserDTO get_user_by_id(int user_id);
```

Expected behavior:

- Return user when found.
- Return not found when ID does not exist.

Cluster 2: Products, Cart, and Dashboard

Owner focus: item HashMap, product management, cart state, stock, and dashboard summary outputs.

GET /products

Optional query:

```
GET /products?category=Electronics
```

Response:

```
[
  {
    "id": 101,
    "name": "Wireless Keyboard",
    "price": 1650,
    "category": "Electronics",
    "popularity_score": 91,
    "stock": 12,
    "image": "https://example.com/image.jpg",
    "description": "Low-profile wireless keyboard."
  }
]
```

Python wrapper:

```
def list_products(category: str | None = None) -> list[dict]:
    return cpp_core.list_products(category)
```

C++ function to expose:

```
std::vector<ProductDTO> list_products(const std::string& category);
```

Expected behavior:

- If `category` is empty, return all products.
- If `category` is supplied, return products in that category only.

GET `/products/{item_id}`

Response:

```
{
  "id": 101,
  "name": "Wireless Keyboard",
  "price": 1650,
  "category": "Electronics",
  "popularity_score": 91,
  "stock": 12,
  "image": "https://example.com/image.jpg",
  "description": "Low-profile wireless keyboard."
}
```

Python wrapper:

```
def get_product(item_id: int) -> dict:
    return cpp_core.get_product_by_id(item_id)
```

C++ function to expose:

```
ProductDTO get_product_by_id(int item_id);
```

Expected behavior:

- Return product when found.
- Return not found when product ID does not exist.

POST `/products`

Request:

```
{
  "name": "Cold Brew Kit",
  "price": 690,
  "category": "Kitchen",
  "stock": 22,
  "image": "https://example.com/image.jpg",
  "description": "Simple glass cold brew maker.",
  "popularity_score": 40
}
```

Response:

```
{
  "id": 109,
  "name": "Cold Brew Kit",
  "price": 690,
  "category": "Kitchen",
  "popularity_score": 40,
  "stock": 22,
  "image": "https://example.com/image.jpg",
  "description": "Simple glass cold brew maker."
}
```

Python wrapper:

```
def create_product(payload: dict) -> dict:
    return cpp_core.create_product(payload)
```

C++ function to expose:

```
ProductDTO create_product(const ProductInput& input);
```

Expected behavior:

- Assign a new product ID.
- Insert product into the item store.
- Insert product into ranking heap.
- Add graph node if graph module requires explicit nodes.

PUT /products/{item_id}

Request:

```
{
  "name": "Wireless Keyboard",
  "price": 1650,
  "category": "Electronics",
  "stock": 12,
  "image": "https://example.com/image.jpg",
  "description": "Low-profile wireless keyboard.",
  "popularity_score": 91
}
```

Response:

```
{
  "id": 101,
  "name": "Wireless Keyboard",
  "price": 1650,
  "category": "Electronics",
  "popularity_score": 91,
  "stock": 12,
  "image": "https://example.com/image.jpg",
  "description": "Low-profile wireless keyboard."
}
```

Python wrapper:

```
def update_product(item_id: int, payload: dict) -> dict:
    return cpp_core.update_product(item_id, payload)
```

C++ function to expose:

```
ProductDTO update_product(int item_id, const ProductInput& input);
```

Expected behavior:

- Update stored product.
- Rebuild or update heap entry if popularity score changed.
- Return not found when product ID does not exist.

DELETE /products/{item_id}

Response:

```
{
  "success": true
}
```

Python wrapper:


```
def delete_product(item_id: int) -> dict:
    return {"success": cpp_core.delete_product(item_id)}
```

C++ function to expose:

```
bool delete_product(int item_id);
```

Expected behavior:

- Remove product from item store.
- Remove or invalidate product in ranking heap.
- Remove related cart rows.
- Remove or ignore graph edges for this item.

GET /users/{user_id}/cart

Response:

```
[
  {
    "user_id": 1,
    "item_id": 101,
    "quantity": 2,
    "product": {
      "id": 101,
      "name": "Wireless Keyboard",
      "price": 1650,
      "category": "Electronics",
      "popularity_score": 91,
      "stock": 12
    }
  }
]
```

Python wrapper:

```
def get_cart(user_id: int) -> list[dict]:
    return cpp_core.get_cart(user_id)
```

C++ function to expose:

```
std::vector<CartItemDTO> get_cart(int user_id);
```

POST /users/{user_id}/cart

Request:

```
{
  "item_id": 101,
  "quantity": 1
}
```

Response:

```
[
  {
    "user_id": 1,
    "item_id": 101,
    "quantity": 1,
    "product": {
      "id": 101,
      "name": "Wireless Keyboard",
      "price": 1650
    }
  }
]
```

Python wrapper:

```
def add_to_cart(user_id: int, item_id: int, quantity: int) -> list[dict]:
    cpp_core.add_to_cart(user_id, item_id, quantity)
    cpp_core.record_interaction(user_id, item_id, "cart")
    return cpp_core.get_cart(user_id)
```

C++ functions to expose:

```
bool add_to_cart(int user_id, int item_id, int quantity);
std::vector<CartItemDTO> get_cart(int user_id);
```

Expected behavior:

- Add new cart row or increase quantity.
- Python or C++ must also record a cart interaction.
- Return not found if user or product does not exist.

DELETE /users/{user_id}/cart/{item_id}

Response:

```
[
  {
    "user_id": 1,
    "item_id": 106,
    "quantity": 1,
    "product": {
      "id": 106,
      "name": "Wireless Keyboard"
    }
  }
]
```

Python wrapper:

```
def remove_from_cart(user_id: int, item_id: int) -> list[dict]:
    cpp_core.remove_from_cart(user_id, item_id)
    return cpp_core.get_cart(user_id)
```

C++ functions to expose:

```
bool remove_from_cart(int user_id, int item_id);
std::vector<CartItemDTO> get_cart(int user_id);
```

POST /users/{user_id}/checkout

Response:

```
{
  "success": true,
  "purchased": 2
}
```

Python wrapper:

```
def checkout(user_id: int) -> dict:
    return cpp_core.checkout(user_id)
```

C++ function to expose:

```
CheckoutResult checkout(int user_id);
```

Expected behavior:

- Convert all current cart rows into `purchase` interactions.
- Decrease stock if stock tracking is enabled.
- Clear cart for the user.
- Return number of purchased rows/items.

GET `/dashboard/stats`

Response:

```
{
  "totalProducts": 8,
  "totalUsers": 3,
  "totalInteractions": 4,
  "mostPopularCategory": "Electronics"
}
```

Python wrapper:

```
def get_dashboard_stats() -> dict:
    return cpp_core.get_dashboard_stats()
```

C++ function to expose:

```
DashboardStats get_dashboard_stats();
```

Cluster 3: Interactions and Recommendations

Owner focus: queue, heap ranking, graph similarity, scoring engine, and behavior updates.

POST `/interactions`

Request:

```
{
  "user_id": 1,
  "item_id": 101,
  "type": "view"
}
```

Response:

```
{
  "id": 5,
  "user_id": 1,
  "item_id": 101,
  "type": "view",
  "timestamp": "2026-05-05T10:30:00Z"
}
```

Python wrapper:

```
def record_interaction(user_id: int, item_id: int, interaction_type: str) -> dict:
    return cpp_core.record_interaction(user_id, item_id, interaction_type)
```

C++ function to expose:

```
InteractionDTO record_interaction(int user_id,
                                int item_id,
                                const std::string& interaction_type);
```

Expected behavior:

- Validate user exists.
- Validate product exists.
- Enqueue interaction.
- Log/store interaction.
- Update user activity score.
- Update item popularity score.
- Update heap ranking.
- Optionally update graph edge weights based on co-interactions.

GET /interactions/recent

Response:

```
[
  {
    "id": 5,
    "user_id": 1,
    "item_id": 101,
    "type": "view",
    "timestamp": "2026-05-05T10:30:00Z"
  }
]
```

Python wrapper:

```
def get_recent_interactions(limit: int = 8) -> list[dict]:
    return cpp_core.get_recent_interactions(limit)
```

C++ function to expose:

```
std::vector<InteractionDTO> get_recent_interactions(int limit);
```

Expected behavior:

- Return newest interactions first.
- Respect limit.

GET /recommendations/{user_id}

Optional query:

```
GET /recommendations/1?limit=4
```

Response:

```
[
  {
    "id": 106,
    "name": "Wireless Keyboard",
    "price": 1650,
    "category": "Electronics",
    "popularity_score": 91,
    "stock": 12,
    "recommendation_score": 118.5,
    "reason": "Because you interacted with Electronics products"
  }
]
```

Python wrapper:

```
def get_recommendations(user_id: int, limit: int = 4) -> list[dict]:
    return cpp_core.get_recommendations(user_id, limit)
```

C++ function to expose:

```
std::vector<RecommendationDTO> get_recommendations(int user_id, int limit);
```

Expected behavior:

- For unknown or new users, return global trending products.
- For known users, combine:
 - item popularity score
 - graph similarity to recently interacted products
 - category/user interest boost
 - optional recency weight
- Exclude unavailable products when `stock <= 0`.
- Return sorted highest score first.

GET /products/{item_id}/related

Optional query:

```
GET /products/101/related?limit=3
```

Response:

```
[
  {
    "id": 106,
    "name": "Wireless Keyboard",
    "price": 1650,
    "category": "Electronics",
    "popularity_score": 91,
    "stock": 12,
    "similarity_score": 0.87
  }
]
```

Python wrapper:

```
def get_related_products(item_id: int, limit: int = 3) -> list[dict]:
    return cpp_core.get_related_products(item_id, limit)
```

C++ function to expose:

```
std::vector<RelatedProductDTO> get_related_products(int item_id, int limit);
```

Expected behavior:

- Use graph neighbors first.
- Sort by similarity weight, then popularity.
- Fallback to same-category products if graph has no neighbors.

GET /trending

Optional query:

```
GET /trending?limit=8
```

Response:

```
[
  {
    "id": 101,
    "name": "AirPods Pro MaxCase",
    "price": 2499,
    "category": "Electronics",
    "popularity_score": 94,
    "stock": 18
  }
]
```

Python wrapper:

```
def get_trending(limit: int = 8) -> list[dict]:
    return cpp_core.get_trending(limit)
```

C++ function to expose:

```
std::vector<ProductDTO> get_trending(int limit);
```

Expected behavior:

- Use max heap or ranking structure.
- Return highest popularity products first.

Minimum C++ Functions Needed For Pybind

The Python layer can be built cleanly if C++ exposes this exact function set:

Cluster 1

- `authenticate_user(email, password)`
- `create_user(input)`
- `get_user_by_id(user_id)`

Cluster 2

- `list_products(category)`
- `get_product_by_id(item_id)`
- `create_product(input)`
- `update_product(item_id, input)`
- `delete_product(item_id)`
- `get_cart(user_id)`
- `add_to_cart(user_id, item_id, quantity)`
- `remove_from_cart(user_id, item_id)`
- `checkout(user_id)`
- `get_dashboard_stats()`

Cluster 3

- `record_interaction(user_id, item_id, interaction_type)`
- `get_recent_interactions(limit)`
- `get_recommendations(user_id, limit)`
- `get_related_products(item_id, limit)`
- `get_trending(limit)`

Suggested Pybind Module Shape

Expose one module named `cpp_core`.

```

PYBIND11_MODULE(cpp_core, m) {
    m.def("authenticate_user", &authenticate_user);
    m.def("create_user", &create_user);
    m.def("get_user_by_id", &get_user_by_id);

    m.def("list_products", &list_products);
    m.def("get_product_by_id", &get_product_by_id);
    m.def("create_product", &create_product);
    m.def("update_product", &update_product);
    m.def("delete_product", &delete_product);
    m.def("get_cart", &get_cart);
    m.def("add_to_cart", &add_to_cart);
    m.def("remove_from_cart", &remove_from_cart);
    m.def("checkout", &checkout);
    m.def("get_dashboard_stats", &get_dashboard_stats);

    m.def("record_interaction", &record_interaction);
    m.def("get_recent_interactions", &get_recent_interactions);
    m.def("get_recommendations", &get_recommendations);
    m.def("get_related_products", &get_related_products);
    m.def("get_trending", &get_trending);
}

```

Python Layer Rule

FastAPI routes should not know about C++ classes directly. Keep that knowledge in a Python service wrapper.

Recommended Python layout:

```

backend/
  main.py
  schemas.py
  services/
    auth_service.py
    product_service.py
    interaction_service.py

```

Each service imports `cpp_core`, calls the masked C++ function, and converts the result into a JSON-ready dict/list.

Frontend Compatibility Notes

The frontend currently expects these `api.js` function names:

- `login`
- `signup`
- `getProducts`
- `getItem`
- `getItemsByCategory`
- `addProduct`
- `updateProduct`
- `deleteProduct`
- `recordInteraction`
- `getRecentInteractions`
- `getRecommendations`
- `getRelatedProducts`
- `getCart`
- `addToCart`
- `removeFromCart`
- `checkout`
- `getDashboardStats`

FastAPI can use snake_case internally, but responses must match the frontend data shape above.

Error Contract

Use standard HTTP status codes:

- `400`: invalid request body or unsupported interaction type

- 401 : invalid login
- 404 : user or product not found
- 409 : duplicate email
- 500 : unexpected C++ or binding error

Example error response:

```
{
  "detail": "Product not found"
}
```

Handoff Checklist For C++ Team

- ☐ Implement DTO structs or return types that pybind can convert cleanly.
- ☐ Keep exposed function names stable.
- ☐ Do not expose raw pointers across the pybind boundary.
- ☐ Return copies or value objects, not internal storage references.
- ☐ Validate IDs before mutating state.
- ☐ Keep password fields out of public return objects.
- ☐ Update item popularity after every interaction.
- ☐ Update user activity score after every interaction.
- ☐ Keep heap ranking consistent after score changes.
- ☐ Keep graph related-products function deterministic.
- ☐ Provide small C++ tests for each exposed function.
- ☐ Provide seed data loader or initialization function if needed.

Handoff Checklist For Python Team

- ☐ Build Pydantic schemas matching the request/response shapes.
- ☐ Create FastAPI routers by cluster.
- ☐ Import `cpp_core` only inside service modules.
- ☐ Translate C++ errors/results into HTTP exceptions.
- ☐ Convert C++ DTOs into dict/list responses.
- ☐ Keep endpoint paths aligned with this document.
- ☐ Add CORS for the React frontend.
- ☐ Add tests for each FastAPI route using mocked service functions.